HRT CoderPad — 60-Minute C++ Coding Interview

Production-Quality Code + Explanations + Extensions + Trading Context

Coverage & Format

Each question includes: full production-quality solution | what HRT expects you to say | follow-up extension | trading system context

Category 1 RAII (Q1-2): Generic RAII handle, thread-safe object pool with lease semantics
Category 2 Lock-Free (Q3-4): MPSC queue, Treiber stack with hazard pointer reclamation
Category 3 Concurrency (Q5-6): Work-stealing thread pool, RCU config store
Category 4 Templates (Q7-8): Type-safe trading units, compile-time ring buffer
Category 5 Trading (Q9-10): L2 order book with O(1) BBO, TWAP execution engine
Category 6 STL (Q11-12): K-largest streaming, VWAP computation

CATEGORY 1 — RAII, Smart Pointers & Resource Management

Q1. Implement a generic RAII handle wrapper for C-API resources (e.g., file descriptors, sockets). [RAII] ~15 min

Approach:

Design a move-only RAII wrapper that: acquires in constructor, releases in destructor, supports custom deleters via template policy, exposes get()/release()/reset(), and works with STL containers. Show Rule of Five. Demonstrate with a file descriptor.

Full Solution:

```
// Production-quality RAII handle for any C-API resource
template<typename T, typename Deleter>
class Handle {
    T       value_;
    Deleter deleter_;
    bool    owned_ = true;

public:
    // Sentinel: value representing "no resource"
    static constexpr T null_value() {
        if constexpr (std::is_pointer_v<T>) return nullptr;
        else return T{-1};
    }

    // Construct: take ownership
    explicit Handle(T v, Deleter d = Deleter{})
        : value_(v), deleter_(std::move(d)), owned_(v != null_value()) {}

    // Destructor: release resource
    ~Handle() {
        if (owned_ && value_ != null_value())
            deleter_(value_);
    }

    // Move: transfer ownership
    Handle(Handle&& o) noexcept
        : value_(std::exchange(o.value_, null_value())),
          deleter_(std::move(o.deleter_)),
          owned_(std::exchange(o.owned_, false)) {}

    Handle& operator=(Handle&& o) noexcept {
        if (this != &o) {
            reset();
            value_   = std::exchange(o.value_,   null_value());
            deleter_ = std::move(o.deleter_);
            owned_   = std::exchange(o.owned_,   false);
        }
        return *this;
    }

    // No copy
    Handle(const Handle&)            = delete;
    Handle& operator=(const Handle&) = delete;

    // Accessors
    T    get()     const noexcept { return value_; }
    bool valid()   const noexcept { return owned_ && value_ != null_value(); }
    explicit operator bool() const noexcept { return valid(); }

    // Release: give up ownership without destroying
    [[nodiscard]] T release() noexcept {
        owned_ = false;
```

```
        return std::exchange(value_, null_value());
    }

    // Reset: destroy current, optionally take new
    void reset(T v = null_value()) {
        if (owned_ && value_ != null_value()) deleter_(value_);
        value_ = v;
        owned_ = (v != null_value());
    }
};

// ---- Usage examples ----

// File descriptor
struct FDDeleter { void operator()(int fd) const { ::close(fd); } };
using FDHandle = Handle<int, FDDeleter>;

FDHandle openLog(const char* path) {
    int fd = ::open(path, O_RDWR | O_CREAT | O_APPEND, 0644);
    if (fd < 0) throw std::system_error(errno, std::system_category());
    return FDHandle{fd};
}

{
    auto log = openLog("/var/log/trades.log");
    if (log) ::write(log.get(), "tick\n", 5);
} // close() called automatically

// Socket handle (same pattern)
struct SocketDeleter { void operator()(int s) const { ::close(s); } };
using SocketHandle = Handle<int, SocketDeleter>;

// Pointer resource (e.g., SDL_Window*)
struct SDLWinDel { void operator()(SDL_Window* w) const { SDL_DestroyWindow(w); } };
using WinHandle = Handle<SDL_Window*, SDLWinDel>;
```

HRT expects: They want to see: explicit operator bool, [[nodiscard]] on release(), noexcept on move, std::exchange pattern, and the null_value() sentinel. Mention that release() is [[nodiscard]] because discarding it causes a leak.

Extension: Add a shared handle (reference-counted) using std::shared_ptr with a custom deleter. Show how to move from Handle<> into a shared_ptr<> without double-delete.

Trading context: HRT wraps every C-API resource (network sockets, memory-mapped files for shared memory IPC, eventfd for cross-thread signalling) in RAII handles like this. The trading system cannot tolerate leaked file descriptors — they exhaust the OS per-process limit of ~65536.

Q2. Implement a thread-safe object pool with RAII lease semantics. Objects must auto-return on scope exit. [RAII] *~20 min*

Approach:

A pool pre-allocates N objects and lends them via RAII Lease objects. On Lease destruction, the object returns to the pool. The pool must be thread-safe (concurrent acquire/release). Use a blocking acquire with optional timeout. Demonstrate with Order objects.

Full Solution:

```
template<typename T>
class ObjectPool {
    std::vector<std::unique_ptr<T>> storage_;   // owns all objects
    std::vector<T*>                 available_; // free list
    mutable std::mutex              mtx_;
    std::condition_variable         cv_;
```

```
public:
    // ---- RAII Lease ----
    class Lease {
        T*          obj_  = nullptr;
        ObjectPool*  pool_ = nullptr;
    public:
        Lease() = default;
        Lease(T* obj, ObjectPool* pool) : obj_(obj), pool_(pool) {}

        ~Lease() { if (obj_) pool_->release(obj_); }

        // Move-only
        Lease(Lease&& o) noexcept
            : obj_(std::exchange(o.obj_, nullptr)),
              pool_(std::exchange(o.pool_, nullptr)) {}

        Lease& operator=(Lease&& o) noexcept {
            if (this != &o) {
                if (obj_) pool_->release(obj_);
                obj_  = std::exchange(o.obj_,  nullptr);
                pool_ = std::exchange(o.pool_, nullptr);
            }
            return *this;
        }

        Lease(const Lease&)            = delete;
        Lease& operator=(const Lease&) = delete;

        T*          get()   const { return obj_; }
        T&          operator*()  const { return *obj_; }
        T*          operator->() const { return obj_; }
        explicit operator bool() const { return obj_ != nullptr; }
    };

    // ---- Pool construction ----
    explicit ObjectPool(size_t n) {
        storage_.reserve(n);
        available_.reserve(n);
        for (size_t i = 0; i < n; ++i) {
            storage_.push_back(std::make_unique<T>());
            available_.push_back(storage_.back().get());
        }
    }

    // ---- Acquire (blocking with timeout) ----
    Lease acquire(std::chrono::milliseconds timeout = std::chrono::milliseconds{100}) {
        std::unique_lock lk(mtx_);
        if (!cv_.wait_for(lk, timeout, [this]{ return !available_.empty(); }))
            return {};  // empty lease on timeout
        T* obj = available_.back();
        available_.pop_back();
        return Lease{obj, this};
    }

    // ---- Non-blocking try acquire ----
    Lease tryAcquire() {
        std::lock_guard lk(mtx_);
        if (available_.empty()) return {};
        T* obj = available_.back();
        available_.pop_back();
        return Lease{obj, this};
    }

    size_t available() const { std::lock_guard lk(mtx_); return available_.size(); }
    size_t total()     const { return storage_.size(); }
```

```
private:
    friend class Lease;
    void release(T* obj) {
        std::lock_guard lk(mtx_);
        available_.push_back(obj);
        cv_.notify_one();
    }
};

// ---- Usage ----
struct Order { int64_t price; int32_t qty; char side; };

ObjectPool<Order> orderPool{1024};

void processNewOrder(int64_t price, int32_t qty) {
    auto lease = orderPool.acquire(std::chrono::milliseconds{50});
    if (!lease) {
        std::cerr << "Pool exhausted!\n";
        return;
    }
    lease->price = price;
    lease->qty   = qty;
    lease->side  = 'B';
    submitOrder(*lease);
} // order returns to pool here automatically
```

HRT expects: Key points: the Lease holds raw pointer (not owning), pool holds unique_ptr (owning). cv_.notify_one() in release() wakes blocked acquirers. [[nodiscard]] could be added to acquire() — mention it. Discuss ABA-safety: pool objects are never freed, only recycled.

Extension: Make it lock-free using an atomic free-list with compare_exchange. Show how this eliminates the mutex and condition variable for the hot path.

Trading context: HRT pre-allocates pools of Order, Message, and BookLevel objects at startup. On the hot path (feed handler receives a message, allocates from pool), there must be zero heap allocations — malloc has an internal lock and can take microseconds.

CATEGORY 2 — Lock-Free Data Structures

Q3. Implement a lock-free MPSC (multi-producer, single-consumer) queue. Explain why MPSC is easier than MPMC. [Lock-Free] *~25 min*

Approach:

MPSC: multiple producers CAS on the tail (to claim a slot), one consumer pops from head without synchronisation (no competing consumers). Use a Michael-Scott queue variant with a sentinel node. MPSC is simpler than MPMC because the consumer has exclusive access to the head — no CAS needed on dequeue.

Full Solution:

```
// Lock-free MPSC queue using linked nodes
// Multiple threads can push(), only ONE thread calls pop()

template<typename T>
class MPSCQueue {
    struct Node {
        std::optional<T>    data;
        std::atomic<Node*>  next{nullptr};
        Node() = default;
```

```
        explicit Node(T v) : data(std::move(v)) {}
    };

    // head_: owned by CONSUMER only (no atomics needed on consumer side)
    Node* head_;
    // tail_: shared by all producers (atomic CAS)
    alignas(64) std::atomic<Node*> tail_;

public:
    MPSCQueue() {
        // Sentinel node: head and tail both point to it initially
        auto* sentinel = new Node{};
        head_ = sentinel;
        tail_.store(sentinel, std::memory_order_relaxed);
    }

    ~MPSCQueue() {
        // Drain remaining nodes
        T val;
        while (pop(val)) {}
        delete head_; // delete sentinel
    }

    // push(): called by MULTIPLE producer threads
    void push(T val) {
        Node* node = new Node{std::move(val)};
        // Atomically swing tail to the new node
        // exchange is atomic: returns old tail
        Node* prev = tail_.exchange(node, std::memory_order_acq_rel);
        // Link old tail to new node
        // This is the only "window" where queue may appear empty
        // but it is momentary (producer will complete the link)
        prev->next.store(node, std::memory_order_release);
    }

    // pop(): called by SINGLE consumer thread ONLY
    bool pop(T& out) {
        Node* next = head_->next.load(std::memory_order_acquire);
        if (!next) return false;  // queue is empty
        out = std::move(*next->data);
        delete head_;   // free old sentinel
        head_ = next;   // next becomes new sentinel
        next->data.reset();
        return true;
    }

    bool empty() const {
        return head_->next.load(std::memory_order_acquire) == nullptr;
    }
};

// ---- Why MPSC is easier than MPMC ----
// MPSC: pop() needs no CAS (single consumer, no race on head)
// MPMC: both push and pop need CAS, plus ABA problem on both ends

// ---- SPSC (even simpler): ring buffer, no heap allocation ----
template<typename T, size_t Cap>
class SPSCQueue {
    static_assert((Cap & (Cap-1)) == 0);
    alignas(64) std::atomic<size_t> write_{0};
    alignas(64) std::atomic<size_t> read_ {0};
    T buf_[Cap];
public:
    bool push(T v) {
        size_t w = write_.load(std::memory_order_relaxed);
        if (w - read_.load(std::memory_order_acquire) >= Cap) return false;
        buf_[w & (Cap-1)] = std::move(v);
```

```
        write_.store(w + 1, std::memory_order_release);
        return true;
    }
    bool pop(T& out) {
        size_t r = read_.load(std::memory_order_relaxed);
        if (r == write_.load(std::memory_order_acquire)) return false;
        out = std::move(buf_[r & (Cap-1)]);
        read_.store(r + 1, std::memory_order_release);
        return true;
    }
};
```

HRT expects: Be prepared to explain: (1) why exchange is used instead of CAS in push() — it makes the push atomic and wait-free for the tail swing; (2) the "linking window" where the queue briefly appears empty to the consumer; (3) why the consumer does not need atomics for head_ — it is the only thread accessing it. Mention memory ordering explicitly.

Extension: Add batch pop (pop_all) that drains all available items in one pass, returning them in a vector. Discuss when batching improves throughput.

Trading context: HRT uses MPSC queues for risk management: multiple strategy threads push orders to a single risk-check thread. The risk thread pops at high frequency without any lock overhead. A blocking dequeue on the risk thread would stall all strategy threads.

Q4. Implement a lock-free atomic stack (Treiber stack). Add hazard pointer-based memory reclamation. [Lock-Free] *~20 min*

Approach:

The Treiber stack uses a single atomic top pointer. push: CAS top from old_top to new_node. pop: CAS top from old_top to old_top->next. Core challenge: memory reclamation — a thread cannot free a popped node because another thread may still be reading it. Hazard pointers publish "I am using this pointer" before dereferencing.

Full Solution:

```
// Treiber Stack with simplified hazard pointer reclamation

template<typename T>
class TreiberStack {
    struct Node {
        T     data;
        Node* next = nullptr;
        explicit Node(T v) : data(std::move(v)) {}
    };

    std::atomic<Node*> top_{nullptr};

    // ---- Simplified hazard pointer (1 per thread) ----
    static constexpr int MAX_THREADS = 64;
    static inline std::atomic<Node*> hazards_[MAX_THREADS] = {};
    static inline std::atomic<int>   threadCount_{0};
    static thread_local int          tid_;

    static int getThreadId() {
        static thread_local int id = threadCount_.fetch_add(1);
        return id;
    }

    void retireNode(Node* node) {
        // Check if any thread has this node as hazard
        for (int i = 0; i < MAX_THREADS; ++i) {
            if (hazards_[i].load(std::memory_order_acquire) == node)
                return; // defer: another thread is reading this node
```

```
        }
        delete node; // safe to free
    }

public:
    void push(T val) {
        Node* node = new Node{std::move(val)};
        Node* old;
        do {
            old = top_.load(std::memory_order_relaxed);
            node->next = old;
        } while (!top_.compare_exchange_weak(old, node,
                  std::memory_order_release,
                  std::memory_order_relaxed));
    }

    std::optional<T> pop() {
        int tid = getThreadId();
        Node* old;
        while (true) {
            old = top_.load(std::memory_order_acquire);
            if (!old) return std::nullopt; // empty

            // Publish hazard: "I am about to read old"
            hazards_[tid].store(old, std::memory_order_seq_cst);

            // Re-validate: top_ may have changed since load
            if (top_.load(std::memory_order_acquire) != old) {
                hazards_[tid].store(nullptr, std::memory_order_release);
                continue; // retry
            }

            // Try to remove old from stack
            if (top_.compare_exchange_strong(old, old->next,
                    std::memory_order_release,
                    std::memory_order_relaxed)) {
                hazards_[tid].store(nullptr, std::memory_order_release);
                T val = std::move(old->data);
                retireNode(old); // free if no other hazard
                return val;
            }
            hazards_[tid].store(nullptr, std::memory_order_release);
        }
    }

    bool empty() const {
        return top_.load(std::memory_order_acquire) == nullptr;
    }
};
```

HRT expects: HRT will probe: (1) Why not just delete the node immediately after pop? (ABA: another thread loaded the pointer before CAS succeeded and is about to dereference it). (2) Why seq_cst on the hazard store? (Must be visible to all threads scanning hazards before they decide to free). (3) What is the ABA problem here? (A node could be popped, freed, reallocated at the same address, and pushed back — old CAS sees the same address and wrongly succeeds).

Extension: Replace deletion with epoch-based reclamation: three epochs, retire nodes to the current epoch, advance epoch when all threads have passed through a quiescent state.

Trading context: HRT uses lock-free stacks for the per-thread free list in a pool allocator. When a thread finishes processing a message, it pushes the node back to the free list — the push must be wait-free (no spinning, no blocking) to guarantee bounded latency.

CATEGORY 3 — Concurrency Patterns

Q5. Implement a thread pool with work stealing, futures for result retrieval, and graceful shutdown. [Concurrency] ~20 min

Approach:

Each worker has a local deque. Submission pushes to the submitting thread's local deque. When a worker's local deque is empty, it steals from a random other worker's back. Graceful shutdown: stop flag + join all workers + process remaining tasks in queue.

Full Solution:

```
class ThreadPool {
    using Task = std::function<void()>;

    // Per-worker local queue (deque for stealing from back)
    struct Worker {
        std::deque<Task>    localQ;
        std::mutex          mtx;
        std::atomic<bool>   idle{false};
    };

    std::vector<std::unique_ptr<Worker>> workers_;
    std::vector<std::thread>             threads_;
    std::atomic<bool>                    stop_{false};
    std::atomic<size_t>                  nextWorker_{0};

    void workerLoop(size_t id) {
        auto& me = *workers_[id];
        while (!stop_.load(std::memory_order_relaxed)) {
            Task task;
            // 1. Try local queue first (cache warm)
            {
                std::lock_guard lk(me.mtx);
                if (!me.localQ.empty()) {
                    task = std::move(me.localQ.front());
                    me.localQ.pop_front();
                }
            }
            // 2. Try stealing from a random other worker
            if (!task) {
                size_t victim = (id + 1) % workers_.size();
                std::unique_lock lk(workers_[victim]->mtx,
                                    std::try_to_lock);
                if (lk && !workers_[victim]->localQ.empty()) {
                    task = std::move(workers_[victim]->localQ.back());
                    workers_[victim]->localQ.pop_back(); // steal from back
                }
            }
            if (task) { me.idle = false; task(); }
            else      { me.idle = true; std::this_thread::yield(); }
        }
    }

public:
    explicit ThreadPool(size_t n = std::thread::hardware_concurrency()) {
        workers_.reserve(n);
        threads_.reserve(n);
        for (size_t i = 0; i < n; ++i) {
            workers_.push_back(std::make_unique<Worker>());
            threads_.emplace_back(&ThreadPool::workerLoop, this, i);
        }
    }
```

```
    // Submit with future return
    template<typename F, typename... Args>
    auto submit(F&& f, Args&&... args) {
        using Ret = std::invoke_result_t<F, Args...>;
        auto task = std::make_shared<std::packaged_task<Ret()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...));
        std::future<Ret> fut = task->get_future();

        // Round-robin to a worker's local queue
        size_t id = nextWorker_.fetch_add(1) % workers_.size();
        {
            std::lock_guard lk(workers_[id]->mtx);
            workers_[id]->localQ.push_back([task]{ (*task)(); });
        }
        return fut;
    }

    // Graceful shutdown: finish all tasks then stop
    ~ThreadPool() {
        stop_.store(true, std::memory_order_release);
        for (auto& t : threads_)
            if (t.joinable()) t.join();
    }
};

// ---- Usage ----
ThreadPool pool{4};

auto f1 = pool.submit([](int x){ return x*x; }, 7);
auto f2 = pool.submit([](){ return computeVWAP(); });

std::cout << f1.get(); // 49
std::cout << f2.get(); // VWAP result
```

HRT expects: HRT expects: noexcept on move, stop_ with memory_order_release in destructor, mention that std::function has overhead (consider templated submit), and discuss when work stealing actually helps vs. hurts (cache thrashing if workers steal frequently).

Extension: Add task priorities: each worker has a priority_queue instead of deque. High-priority tasks (e.g., risk checks) run before low-priority (e.g., reporting).

Trading context: HRT's analytical compute engine (Greeks calculation, risk attribution) uses a work-stealing pool — each strategy's compute is independent and can be stolen by idle cores. The trading decision path itself is NEVER pooled — it runs on a dedicated pinned thread.

Q6. Implement a read-copy-update (RCU) mechanism for a shared configuration object that supports lock-free reads and atomic updates. [Concurrency] *~15 min*

Approach:

Use an atomic<shared_ptr<Config>> for the snapshot. Readers load the shared_ptr atomically (lock-free read, shared ownership). Writers create a new Config, atomically swap the pointer, then let old Config be destroyed when all readers release their shared_ptr copies. This is a simplified user-space RCU using reference counting.

Full Solution:

```
// RCU for read-heavy shared configuration

struct RiskConfig {
    int64_t maxPosition  = 0;
    int64_t maxNotional  = 0;
    double  maxDrawdown  = 0.0;
    int32_t maxOrderRate = 0;
};
```

```
class RCUConfig {
    // C++20: atomic<shared_ptr<T>> is natively atomic
    std::atomic<std::shared_ptr<const RiskConfig>> current_;

public:
    explicit RCUConfig(RiskConfig cfg)
        : current_(std::make_shared<const RiskConfig>(std::move(cfg))) {}

    // READ: completely lock-free, called millions of times/sec
    // Returns a shared_ptr snapshot -- caller owns it for its duration
    std::shared_ptr<const RiskConfig> read() const {
        return current_.load(std::memory_order_acquire);
    }

    // UPDATE: create new copy, atomically publish
    void update(RiskConfig newCfg) {
        auto newSnap = std::make_shared<const RiskConfig>(std::move(newCfg));
        current_.store(newSnap, std::memory_order_release);
        // Old shared_ptr ref-count decrements here
        // Old RiskConfig freed when last reader releases its copy
    }

    // CONDITIONAL UPDATE: only apply if version matches
    bool compareAndUpdate(std::shared_ptr<const RiskConfig> expected,
                          RiskConfig                        newCfg) {
        auto newSnap = std::make_shared<const RiskConfig>(std::move(newCfg));
        return current_.compare_exchange_strong(expected, newSnap,
                   std::memory_order_acq_rel,
                   std::memory_order_acquire);
    }
};

// ---- Usage ----
RCUConfig riskCfg{RiskConfig{1000000, 10000000, 0.05, 100}};

// Strategy thread (reads millions of times per second):
void checkRisk(const Order& order) {
    auto cfg = riskCfg.read(); // lock-free!
    if (order.qty > cfg->maxPosition)
        throw RiskBreach{"position limit"};
    if (order.qty * order.price > cfg->maxNotional)
        throw RiskBreach{"notional limit"};
    // cfg shared_ptr released here -> old config freed if no other readers
}

// Config thread (rarely, on config change):
void onConfigChange(const NewConfig& nc) {
    RiskConfig updated = loadFromDB(nc);
    riskCfg.update(std::move(updated));
    // All subsequent readers see new config; old freed when ready
}

// ---- Why this works ----
// read() returns a shared_ptr with its own ref-count increment
// Even if update() runs mid-read, the old config stays alive
// until ALL readers have released their shared_ptr copies
```

HRT expects: Key insight: atomic<shared_ptr<T>> (C++20) provides the atomic load/store. Before C++20, use atomic_load/atomic_store free functions. The old config is RAII-managed — no explicit epoch tracking needed because shared_ptr IS the reference counter. Discuss trade-off: shared_ptr ref-count updates use atomic ops, so very high read frequency can still cause cache coherence traffic.

Extension: Add a versioned reader: each reader captures the version at read time, and the writer can query "are all readers past version N?" to know when it is safe to free. This is closer to true RCU.

Trading context: HRT's risk parameter store (position limits, notional limits, drawdown limits) is read on every single order submission — potentially 1M times/second. Any locking would serialise all strategy threads. RCU gives lock-free reads with correct update semantics.

CATEGORY 4 — Templates & Generic Design

Q7. Implement a compile-time type-safe unit system for trading quantities (price, quantity, notional) that prevents mixing incompatible types. [Templates] *~20 min*

Approach:

Use a phantom type parameter to tag numeric types as Price, Quantity, or Notional. Overload arithmetic operators so Price * Quantity = Notional, but Price + Quantity is a compile error. Use strong typedef pattern with template class.

Full Solution:

```
// Compile-time type-safe trading quantities

// Tags: cannot be instantiated, just used as phantom types
struct PriceTag    {};
struct QuantityTag {};
struct NotionalTag {};

template<typename Tag, typename Rep = int64_t>
class Quantity {
    Rep value_;
public:
    explicit constexpr Quantity(Rep v) noexcept : value_(v) {}
    constexpr Rep get() const noexcept { return value_; }

    // Same-type arithmetic: Price + Price = Price (OK)
    constexpr Quantity operator+(Quantity rhs) const noexcept
    { return Quantity{value_ + rhs.value_}; }

    constexpr Quantity operator-(Quantity rhs) const noexcept
    { return Quantity{value_ - rhs.value_}; }

    constexpr bool operator< (Quantity rhs) const noexcept { return value_ <
rhs.value_; }
    constexpr bool operator<=(Quantity rhs) const noexcept { return value_ <=
rhs.value_; }
    constexpr bool operator==(Quantity rhs) const noexcept { return value_ ==
rhs.value_; }

    // Scale: Price * 2 = Price (multiply by raw scalar)
    constexpr Quantity operator*(Rep scalar) const noexcept
    { return Quantity{value_ * scalar}; }

    // Negation
    constexpr Quantity operator-() const noexcept { return Quantity{-value_}; }
};

// Type aliases
using Price    = Quantity<PriceTag>;
using Qty      = Quantity<QuantityTag>;
using Notional = Quantity<NotionalTag>;
```

```
// Cross-type multiplication: Price * Qty = Notional
constexpr Notional operator*(Price p, Qty q) noexcept {
    return Notional{p.get() * q.get()};
}
constexpr Notional operator*(Qty q, Price p) noexcept {
    return p * q;
}

// UDLs for readability
constexpr Price    operator""_px(unsigned long long v) { return
Price{static_cast<int64_t>(v)}; }
constexpr Qty      operator""_sh(unsigned long long v) { return
Qty{static_cast<int64_t>(v)}; }

// ---- Usage ----
Price  bid  = 10250_px;
Price  ask  = 10275_px;
Qty    size = 100_sh;

Price  spread    = ask - bid;           // OK: Price - Price = Price
Notional notional = bid * size;          // OK: Price * Qty = Notional
Notional doubled  = notional * 2;        // OK: Notional * scalar

// COMPILE ERRORS -- type safety at work:
// Price bad1 = bid + size;              // ERROR: Price + Qty undefined
// Notional bad2 = bid + notional;       // ERROR: Price + Notional undefined
// auto bad3 = size * size;              // ERROR: Qty * Qty undefined

// Works with STL:
std::vector<Price> prices = {10250_px, 10275_px, 10300_px};
std::sort(prices.begin(), prices.end(), [](Price a, Price b){ return a < b; });

// Compile-time constants:
static_assert(Price{100} + Price{50} == Price{150});
static_assert(Price{100} * Qty{200}  == Notional{20000});
```

HRT expects: Discuss: (1) why the Rep type should be fixed-point int64_t rather than double (precision, no rounding error, deterministic); (2) add explicit to prevent accidental integer->Price conversion; (3) the UDL suffix makes code readable without losing type safety; (4) constexpr means all computations can happen at compile time for static configuration.

Extension: Add a currency dimension (USD, EUR, JPY) as a second phantom type, so USD_Price * Qty = USD_Notional, preventing cross-currency arithmetic without explicit conversion.

Trading context: HRT uses phantom types extensively. A historical bug that motivated this: a function accidentally passed a price in ticks (integer) where a notional in dollars was expected — both were int64_t. The compiler could not catch it. Phantom types eliminate this entire class of bugs at compile time, with zero runtime overhead.

Q8. Implement a compile-time fixed-size ring buffer with template capacity, supporting both trivial and non-trivial types correctly. [Templates] *~15 min*

Approach:

Use aligned_storage for non-trivially-constructible types to avoid default-constructing all slots. Use placement new for emplace, explicit destructor calls for pop. For trivial types, use a plain array. Use if constexpr to select strategy. Show correct head/tail management with power-of-2 masking.

Full Solution:

```
template<typename T, size_t Cap>
class RingBuf {
    static_assert(Cap > 0 && (Cap & (Cap-1)) == 0,
        "Cap must be a power of 2");
```

```
    static constexpr size_t MASK = Cap - 1;

    // Storage: aligned raw bytes for non-trivial types
    // avoids default-constructing all slots
    alignas(T) std::byte storage_[Cap * sizeof(T)];
    size_t head_ = 0;  // consumer index
    size_t tail_ = 0;  // producer index (tail - head = size)

    T* ptr(size_t idx) noexcept {
        return std::launder(reinterpret_cast<T*>(
            &storage_[(idx & MASK) * sizeof(T)]));
    }
    const T* ptr(size_t idx) const noexcept {
        return std::launder(reinterpret_cast<const T*>(
            &storage_[(idx & MASK) * sizeof(T)]));
    }

public:
    RingBuf() = default;

    ~RingBuf() {
        // Destroy live elements
        if constexpr (!std::is_trivially_destructible_v<T>) {
            while (!empty()) {
                ptr(head_)->~T();
                ++head_;
            }
        }
    }

    // No copy (would need to copy-construct elements)
    RingBuf(const RingBuf&) = delete;

    bool empty() const noexcept { return head_ == tail_; }
    bool full()  const noexcept { return (tail_ - head_) == Cap; }
    size_t size()const noexcept { return tail_ - head_; }

    // push by copy
    bool push(const T& val) {
        if (full()) return false;
        new (ptr(tail_)) T(val);  // placement new
        ++tail_;
        return true;
    }

    // push by move
    bool push(T&& val) {
        if (full()) return false;
        new (ptr(tail_)) T(std::move(val));
        ++tail_;
        return true;
    }

    // emplace: perfect-forward construction arguments
    template<typename... Args>
    bool emplace(Args&&... args) {
        if (full()) return false;
        new (ptr(tail_)) T(std::forward<Args>(args)...);
        ++tail_;
        return true;
    }

    // pop: return value, destroy slot
    std::optional<T> pop() {
        if (empty()) return std::nullopt;
        T* p = ptr(head_);
```

```
        T  val = std::move(*p);
        p->~T();   // explicit destructor call
        ++head_;
        return val;
    }

    // peek: view top without removing
    const T& front() const { return *ptr(head_); }
    T&       front()       { return *ptr(head_); }
};

// ---- Test with trivial and non-trivial types ----
RingBuf<int, 8> intBuf;
intBuf.push(1); intBuf.push(2); intBuf.push(3);
auto v = intBuf.pop(); // optional<int>{1}

RingBuf<std::string, 4> strBuf;
strBuf.emplace("hello");  // no copy: constructed in place
strBuf.emplace(5, 'x');   // string(5, 'x') = "xxxxx"
```

HRT expects: Critical points to discuss: (1) std::launder after placement new — without it, compiler may return stale cached values for const-member types; (2) explicit destructor call p->~T() — the only correct way to destroy an object whose lifetime was started with placement new; (3) why aligned_storage/aligned byte array rather than T[Cap] — avoids default-constructing all Cap objects upfront; (4) the MASK trick for power-of-2 modulo.

Extension: Make it work as an SPSC queue with two separate atomic indices (write_ and read_) for lock-free concurrent access. Discuss the memory ordering required.

Trading context: HRT's market data ring buffers store parsed message structs. Using aligned_storage means the 65536-slot buffer does not call the struct constructor 65536 times at startup — only actually-used slots are constructed. This avoids a startup latency spike.

CATEGORY 5 — Trading-Specific Design Problems

Q9. Design and implement a consolidated order book (L2 market data) supporting add, modify, cancel order operations with O(log n) per operation and O(1) BBO (best bid/offer).

[Trading] *~25 min*

Approach:

Use std::map for price levels (sorted, O(log n) lookup) and std::unordered_map for order-ID-to-iterator mapping (O(1) cancel/modify without searching by price). Each price level holds total quantity. BBO is always the first/last element of each map. Handle the case where a level empties (must remove from map).

Full Solution:

```
class L2OrderBook {
public:
    struct Level {
        int64_t totalQty = 0;
        int32_t orderCount = 0;
    };

private:
    using BidMap = std::map<int64_t, Level, std::greater<int64_t>>; // desc
    using AskMap = std::map<int64_t, Level>;                         // asc

    BidMap bids_;
    AskMap asks_;
```

```
    struct OrderInfo {
        int64_t price;
        int32_t qty;
        char    side;
    };
    std::unordered_map<uint64_t, OrderInfo> orders_; // id -> info

    void addQtyToLevel(char side, int64_t price, int32_t qty) {
        if (side == 'B') {
            auto& lvl = bids_[price];
            lvl.totalQty += qty;
            lvl.orderCount++;
        } else {
            auto& lvl = asks_[price];
            lvl.totalQty += qty;
            lvl.orderCount++;
        }
    }

    void removeQtyFromLevel(char side, int64_t price, int32_t qty) {
        if (side == 'B') {
            auto it = bids_.find(price);
            if (it == bids_.end()) return;
            it->second.totalQty -= qty;
            it->second.orderCount--;
            if (it->second.totalQty <= 0) bids_.erase(it);
        } else {
            auto it = asks_.find(price);
            if (it == asks_.end()) return;
            it->second.totalQty -= qty;
            it->second.orderCount--;
            if (it->second.totalQty <= 0) asks_.erase(it);
        }
    }

public:
    // Add new order
    bool addOrder(uint64_t id, int64_t price, int32_t qty, char side) {
        if (orders_.count(id)) return false; // duplicate
        orders_[id] = {price, qty, side};
        addQtyToLevel(side, price, qty);
        return true;
    }

    // Cancel order: O(1) lookup + O(log n) level update
    bool cancelOrder(uint64_t id) {
        auto it = orders_.find(id);
        if (it == orders_.end()) return false;
        auto [price, qty, side] = it->second;
        removeQtyFromLevel(side, price, qty);
        orders_.erase(it);
        return true;
    }

    // Modify quantity: difference applied to level
    bool modifyOrder(uint64_t id, int32_t newQty) {
        auto it = orders_.find(id);
        if (it == orders_.end()) return false;
        auto& info = it->second;
        int32_t delta = newQty - info.qty;
        if (side(info) == 'B') bids_[info.price].totalQty += delta;
        else                   asks_[info.price].totalQty += delta;
        info.qty = newQty;
        return true;
    }
```

```
    static char side(const OrderInfo& o) { return o.side; }

    // O(1) BBO access
    std::optional<std::pair<int64_t,Level>> bestBid() const {
        if (bids_.empty()) return std::nullopt;
        return *bids_.begin();
    }
    std::optional<std::pair<int64_t,Level>> bestAsk() const {
        if (asks_.empty()) return std::nullopt;
        return *asks_.begin();
    }

    // Spread in ticks
    std::optional<int64_t> spread() const {
        auto b = bestBid(), a = bestAsk();
        if (!b || !a) return std::nullopt;
        return a->first - b->first;
    }

    // Top N levels
    std::vector<std::pair<int64_t,Level>> topBids(int n) const {
        std::vector<std::pair<int64_t,Level>> result;
        int cnt = 0;
        for (auto& [px, lvl] : bids_) {
            if (cnt++ >= n) break;
            result.emplace_back(px, lvl);
        }
        return result;
    }
};

// ---- Test ----
L2OrderBook book;
book.addOrder(1, 10250, 100, 'B');
book.addOrder(2, 10250, 200, 'B');  // same level: qty=300
book.addOrder(3, 10275, 150, 'S');
book.cancelOrder(1);                // level 10250 qty=200
auto [bidPx, bidLvl] = *book.bestBid();
// bidPx=10250, bidLvl.totalQty=200, bidLvl.orderCount=1
```

HRT expects: HRT will probe: (1) What if the same order ID is added twice? (Check and return false). (2) Thread safety — what if the book is read while being updated? (Add shared_mutex or use single-writer design). (3) Memory layout — could you replace std::map with a sorted vector for better cache performance? (Yes, for static books; dynamic books need the O(log n) insert). (4) What happens at level removal: empty check is crucial to avoid empty levels at BBO.

Extension: Add a sequence number to each update and support snapshotting the book at a specific sequence number. Also add a "match engine" that fills incoming orders against the book.

Trading context: This is literally what HRT's consolidated book (CBook) does. The orders_ hash map is the key insight: it enables O(1) cancel without scanning the price level for the specific order — critical because cancel messages outnumber new orders by 10:1 in modern markets.

Q10. Implement a TWAP execution engine: given a total quantity, execute it in equal slices over a time window, tracking fill quantity, average fill price, and slippage vs arrival price.

[Trading] *~25 min*

Approach:

A TWAP engine divides totalQty into N equal slices across a time window. Each slice fires at regular intervals. Track: totalFilled, totalNotional, arrivalPrice. Slippage = (avgFillPrice - arrivalPrice) for buys. Use std::chrono for scheduling, atomic for thread-safe fill updates, and a timer callback for slice submission.

Full Solution:

```
#include <chrono>
#include <atomic>
#include <functional>

class TWAPEngine {
public:
    struct Config {
        int64_t  totalQty;       // total shares to execute
        int32_t  numSlices;      // how many equal slices
        std::chrono::milliseconds windowMs; // total time window
        char     side;           // B or S
        int64_t  arrivalPrice;   // benchmark: price at start
    };

    struct Stats {
        std::atomic<int64_t>  totalFilled{0};
        std::atomic<int64_t>  totalNotional{0}; // sum of price*qty
        std::atomic<int32_t>  numFills{0};
        std::atomic<int32_t>  slicesSent{0};
        std::atomic<bool>     done{false};

        double avgFillPrice() const {
            int64_t filled = totalFilled.load();
            if (filled == 0) return 0.0;
            return static_cast<double>(totalNotional.load()) / filled;
        }

        double slippageBps(int64_t arrivalPrice, char side) const {
            double avg = avgFillPrice();
            if (avg == 0.0 || arrivalPrice == 0) return 0.0;
            // Buy slippage: paid more than arrival -> positive
            // Sell slippage: received less than arrival -> positive
            double slip = (side == 'B')
                ? (avg - arrivalPrice)
                : (arrivalPrice - avg);
            return (slip / arrivalPrice) * 10000.0; // in bps
        }
    };

private:
    Config  cfg_;
    Stats   stats_;
    int64_t sliceQty_;     // qty per slice
    int64_t remainingQty_; // handles rounding

    using OrderSubmitter = std::function<void(int64_t qty, char side)>;
    OrderSubmitter submit_;

public:
    explicit TWAPEngine(Config cfg, OrderSubmitter submit)
        : cfg_(std::move(cfg)), submit_(std::move(submit)) {
        sliceQty_     = cfg_.totalQty / cfg_.numSlices;
        remainingQty_ = cfg_.totalQty; // track remaining
    }

    // Call this to trigger each slice (from a timer thread)
    void onSliceTick() {
        if (stats_.done.load()) return;

        int32_t sliceNum = stats_.slicesSent.fetch_add(1);
        if (sliceNum >= cfg_.numSlices) {
            stats_.done = true;
            return;
        }

        // Last slice gets remainder to avoid rounding loss
```

```
        int64_t qty = (sliceNum == cfg_.numSlices - 1)
            ? (cfg_.totalQty - sliceQty_ * sliceNum)
            : sliceQty_;

        submit_(qty, cfg_.side);
    }

    // Call when a fill comes back from the exchange
    void onFill(int64_t fillQty, int64_t fillPrice) {
        stats_.totalFilled.fetch_add(fillQty,   std::memory_order_relaxed);
        stats_.totalNotional.fetch_add(fillQty * fillPrice,
                                       std::memory_order_relaxed);
        stats_.numFills.fetch_add(1, std::memory_order_relaxed);
        if (stats_.totalFilled.load() >= cfg_.totalQty)
            stats_.done = true;
    }

    const Stats& stats()  const { return stats_; }
    bool         isDone() const { return stats_.done.load(); }

    // Print execution summary
    void printSummary() const {
        std::cout << "Filled:   " << stats_.totalFilled << " / " << cfg_.totalQty << "
shares\n"
                  << "AvgPrice: " << stats_.avgFillPrice() << "\n"
                  << "Slippage: " << stats_.slippageBps(cfg_.arrivalPrice, cfg_.side) << "
bps\n"
                  << "Fills:    " << stats_.numFills << "\n";
    }
};

// ---- Timer loop (runs in a dedicated thread) ----
void runTWAP(TWAPEngine& engine, int numSlices,
             std::chrono::milliseconds window) {
    auto sliceInterval = window / numSlices;
    auto next = std::chrono::steady_clock::now();

    for (int i = 0; i < numSlices && !engine.isDone(); ++i) {
        engine.onSliceTick();
        next += sliceInterval;
        std::this_thread::sleep_until(next);
    }
}

// ---- Usage ----
TWAPEngine::Config cfg{
    .totalQty    = 100000,
    .numSlices   = 10,
    .windowMs    = std::chrono::minutes{5},
    .side        = 'B',
    .arrivalPrice = 10250,
};

TWAPEngine engine{cfg, [](int64_t qty, char side) {
    std::cout << "Submit " << qty << " " << side << "\n";
}};

// Timer thread fires onSliceTick() every 30 seconds
// Fill callbacks come from exchange: engine.onFill(filledQty, price)
// Final: engine.printSummary()
```

HRT expects: Key discussion points: (1) Why atomic<int64_t> for stats — fills may come from multiple threads (exchange callbacks); (2) Last slice rounding: 100000 / 10 = 10000, but 10 * 10000 = 100000 exactly — show the case where it does not divide evenly; (3) sleep_until vs sleep_for — sleep_until avoids drift accumulation; (4) The submitter is a std::function dependency injection — allows unit testing without an actual exchange.

Extension: Add participation rate: if market volume in the last slice was 50000 shares, only submit min(sliceQty, participationRate * 50000). Add VWAP benchmark tracking.

Trading context: This is directly your TWAP/VWAP work translated to HRT style. They will expect you to know: (1) why fixed-point int64_t for prices instead of double; (2) that slippage is measured in basis points (bps = 0.01%), not absolute; (3) how to handle partial fills (fill < submitted qty — next slice picks up remainder).

CATEGORY 6 — STL Algorithms & Data Structures

Q11. Implement a function that finds the k largest elements from an unsorted stream of integers using a min-heap, using only O(k) memory. [Algorithm] *~15 min*

Approach:

Maintain a min-heap of size k. For each new element: if heap size < k, push it. If heap size == k and element > heap top, pop the min and push the new element. At the end, the heap contains the k largest. O(n log k) time, O(k) space. Use std::priority_queue with greater<int> (min-heap).

Full Solution:

```
#include <queue>
#include <vector>
#include <functional>

// Find k largest elements from a stream using O(k) memory
class KLargest {
    // Min-heap: top() is the smallest of the k largest
    std::priority_queue<int64_t,
                        std::vector<int64_t>,
                        std::greater<int64_t>> minHeap_;
    size_t k_;

public:
    explicit KLargest(size_t k) : k_(k) {}

    // Process one element from stream: O(log k)
    void add(int64_t val) {
        if (minHeap_.size() < k_) {
            minHeap_.push(val);
        } else if (val > minHeap_.top()) {
            minHeap_.pop();   // remove smallest of top-k
            minHeap_.push(val);
        }
        // else: val is too small to be in top-k, ignore
    }

    // Get results (destroys heap)
    std::vector<int64_t> results() {
        std::vector<int64_t> res;
        res.reserve(minHeap_.size());
        while (!minHeap_.empty()) {
            res.push_back(minHeap_.top());
            minHeap_.pop();
        }
        std::sort(res.rbegin(), res.rend()); // descending
        return res;
    }

    // Kth largest (current minimum of the top-k)
```

```
    int64_t kthLargest() const {
        if (minHeap_.size() < k_)
            throw std::runtime_error("not enough elements");
        return minHeap_.top();
    }
};

// ---- One-shot version using nth_element: O(n) ----
std::vector<int64_t> kLargestNthElement(
    std::vector<int64_t> data, size_t k) {
    if (k >= data.size()) return data;
    // Partition so that elements >= pivot are in [0, k)
    std::nth_element(data.begin(), data.begin() + k, data.end(),
                     std::greater<int64_t>{});
    // data[0..k-1] are the k largest (unsorted among themselves)
    data.resize(k);
    std::sort(data.begin(), data.end(), std::greater<int64_t>{});
    return data;
}

// ---- Test ----
KLargest kl{3};
for (int64_t x : {5, 2, 9, 1, 7, 3, 8, 4, 6}) kl.add(x);
auto top3 = kl.results(); // {9, 8, 7}
std::cout << kl.kthLargest(); // 7 (3rd largest)

// Trading: top-3 most active stocks by volume
KLargest topStocks{3};
for (auto& [symbol, vol] : volumes) topStocks.add(vol);
```

HRT expects: Be ready to explain: (1) Why min-heap for k LARGEST? Because we want to efficiently compare new elements against the current threshold (kth largest = heap top). (2) Time complexity: O(n log k) vs O(n log n) for sort — much faster when k << n. (3) When to use nth_element instead: for one-shot (not streaming), nth_element gives O(n) average with no extra memory. (4) Edge cases: k > n (return all), k = 1 (just track max).

Extension: Adapt for a sliding window of the last W trades: when a trade exits the window, we need to remove it from the heap — discuss why this requires a different data structure (ordered set or indexed priority queue).

Trading context: HRT finds the k most liquid instruments for a market-making strategy (by traded volume in the last N minutes). The streaming version handles real-time volume updates as they arrive from the feed handler.

Q12. Given a sequence of trade events (timestamp, symbol, price, qty), compute a running VWAP per symbol using STL, producing output sorted by symbol and timestamp. [STL] *~20 min*

Approach:

Group trades by symbol using unordered_map. Within each symbol, maintain running sums (totalNotional, totalQty) for VWAP. Produce sorted output using map or sort. Show efficient use of structured bindings, ranges (C++20), and custom comparators.

Full Solution:

```
#include <unordered_map>
#include <map>
#include <vector>
#include <algorithm>
#include <numeric>

struct Trade {
    int64_t     timestamp; // nanoseconds since epoch
    std::string symbol;
    int64_t     price;     // fixed-point: price in cents
    int32_t     qty;
};
```

```
struct VWAPState {
    int64_t totalNotional = 0; // sum of price * qty
    int64_t totalQty      = 0;
    int32_t tradeCount    = 0;

    void update(int64_t price, int32_t qty) {
        totalNotional += price * qty;
        totalQty      += qty;
        ++tradeCount;
    }

    double vwap() const {
        if (totalQty == 0) return 0.0;
        return static_cast<double>(totalNotional) / totalQty;
    }
};

struct VWAPResult {
    std::string symbol;
    double      vwap;
    int64_t     totalQty;
    int32_t     tradeCount;
};

// Compute VWAP per symbol from a vector of trades
std::vector<VWAPResult>
computeVWAP(const std::vector<Trade>& trades) {
    // Group by symbol: O(n)
    std::unordered_map<std::string, VWAPState> states;
    states.reserve(trades.size());

    for (const auto& t : trades) {
        states[t.symbol].update(t.price, t.qty);
    }

    // Build results vector: O(k) where k = unique symbols
    std::vector<VWAPResult> results;
    results.reserve(states.size());

    for (auto& [symbol, state] : states) {
        results.push_back({
            symbol,
            state.vwap(),
            state.totalQty,
            state.tradeCount
        });
    }

    // Sort by symbol (alphabetical): O(k log k)
    std::sort(results.begin(), results.end(),
        [](const VWAPResult& a, const VWAPResult& b) {
            return a.symbol < b.symbol;
        });

    return results;
}

// ---- Per-minute rolling VWAP ----
std::map<std::pair<std::string, int64_t>, VWAPState>
rollingVWAP(const std::vector<Trade>& trades,
            int64_t windowNs = 60000000000LL) { // 1 min (60s)
    std::map<std::pair<std::string, int64_t>, VWAPState> result;
    for (const auto& t : trades) {
        int64_t bucket = t.timestamp / windowNs; // minute bucket
        result[{t.symbol, bucket}].update(t.price, t.qty);
    }
```

```
    return result;
}

// ---- C++20 Ranges version ----
#include <ranges>
void printVWAP(const std::vector<Trade>& trades) {
    auto results = computeVWAP(trades);
    for (const auto& r : results) {
        std::cout << r.symbol << ": VWAP=" << r.vwap
                  << " qty=" << r.totalQty << "\n";
    }
}

// ---- Test ----
std::vector<Trade> trades = {
    {1000, "AAPL", 15000, 100},
    {1001, "MSFT", 30000, 200},
    {1002, "AAPL", 15100, 150},
    {1003, "AAPL", 14900,  50},
    {1004, "MSFT", 30100, 100},
};
auto vwaps = computeVWAP(trades);
// AAPL: (15000*100 + 15100*150 + 14900*50) / 300 = 15016.67
// MSFT: (30000*200 + 30100*100) / 300 = 30033.33
```

HRT expects: Discuss: (1) Why not std::map for the primary grouping? unordered_map is O(1) per insert vs O(log k); (2) reserve() the unordered_map to avoid rehashing; (3) int64_t for totalNotional — price*qty can overflow int32_t; (4) the rollingVWAP version shows how to time-bucket trades for a minute-bar VWAP; (5) structured bindings make the code significantly cleaner.

Extension: Stream version: process trades one at a time from a feed, maintaining running VWAP per symbol. Add a "reset" at the start of each trading day. Handle symbol universe changes (new symbols appearing).

Trading context: VWAP calculation is a core primitive in HRT's analytics stack. Every strategy's execution quality is measured against VWAP. The rolling 1-minute VWAP is computed continuously and used as a signal input for execution algorithms.